

Week1 Homework

By Deng Yufan without AI.

Prob 3

We say that a vertex v in an undirected graph G is an articulation point if its removal increases the number of connected components of G .

a. Try to detect whether v is an articulation point using DFS.

b. Try to find all articulation point using DFS (your algorithm should run in $O(|V| + |E|)$ time). (hint: You can show the following characterization of articulation point. Suppose you are visiting node v in DFS: (1) If v is the root of the DFS tree and it has at least 2 children subgraphs disconnected from each other, then v is an articulation point. (2) If there is no way to reach a node u with strictly smaller discoverytime than that of v following the DFS traversal, then v is an articulation point.)

a. Let $G' = (V \setminus \{v\}, E)$, on G' we do DFS start with unmarked $1, 2, \dots, n$. Each DFS we mark every reached point, and count the times we start a DFS. Then we know #components, and to judge whether greater than 1.

b. We define dfn_i and low_i for each vertex. $dfn_i \in [1, n]$ represents the time that we first visit the vertex; $low_i \in [1, n]$ represents $\min_j$ can be reached by passing atmost 1 non-tree edge dfn_j . For every vertex v , we check all edges connected to v . Suppose the current edge is (v, u) , if u is already visited, then $low_v \leftarrow \min(low_v, dfn_u)$; else DFS through u , and $low_v \leftarrow \min(low_v, low_u)$, count the number of $low_u \geq dfn_v$. If the number is greater than 1 or v is not root, then v is a articulation point.

Prob 4 (P108 T6)

Suppose that an n -node undirected graph $G = (V, E)$ contains two nodes s and t such that the distance between s and t is strictly greater than $n/2$. Show that there must exist some node v , not equal to either s or t , such that deleting v from G destroys all s - t paths. (In other words, the graph obtained from G by deleting v contains no path from s to t .)

Give an algorithm with running time $O(m + n)$ to find such a node v .

For vertex s and t that minimal distance $> n/2$, suppose that there are two paths $s \leftrightarrow_1 t$ and $s \leftrightarrow_2 t$, and $s \leftrightarrow_1 t \cap s \leftrightarrow_2 t = \{s, t\}$, then we find a ring $s \leftrightarrow_1 t \leftrightarrow_2 s$, and it go through $> n$ vertex, contradiction.

We first construct block-cut tree (i.e. 圆方树) for G . Since s and t belong to different VCCs (i.e. 点双), there exists a circle point v (i.e. 圆点) distinct with s and t , and v is on the path between s and t on the block-cut tree. It's well known that v is such a vertex that critical for path between s and t .

Prob 5 (P193 T12)

Suppose you have n video streams that need to be sent, one after another, over a communication link. Stream i consists of a total of b_i bits that need to be sent, at a constant rate, over a period of t_i seconds. You cannot send two streams at the same time, so you need to determine a schedule for the streams: an order in which to send them. Whichever order you choose, there cannot be any delays between the end of one stream and the start of the next. Suppose your schedule starts at time 0 (and therefore ends at time $\sum_{i=1}^n t_i$, whichever order you choose). We assume that all the values b_i and t_i are positive integers.

Now, because you're just one user, the link does not want you taking up too much bandwidth, so it imposes the following constraint, using a fixed parameter r : For each natural number $t > 0$, the total number of bits you send over the time interval from 0 to t cannot exceed rt .

Given a set of n streams, each specified by its number of bits b_i and its time duration t_i , as well as the link parameter r , determine whether there exists a valid schedule.

a. Consider the following claim: There exists a valid schedule if and only if each stream i satisfies $b_i \leq rt_i$. Decide whether you think the claim is true or false, and give a proof of either the claim or its negation.

b. Give an algorithm that takes a set of n streams, each specified by its number of bits b_i and its time duration t_i , as well as the link parameter r , and determines whether there exists a valid schedule. The running time of your algorithm should be polynomial in n .

a. The claim is false. Consider the counter example: $n = 2, r = 2, b_1 = 1, t_1 = 1, b_2 = 3, t_2 = 1$. Since $b_2 > rt_2$, but we can first do job 1 and then do job 2.

b. Only check $\sum_{i=1}^n b_i \leq r \sum_{i=1}^n t_i$ is enough. If the condition is hold, then we have a solution; otherwise no solution.

Now we show the proof of correctness. If the condition is not hold, obviously we will overflow at time $\sum_{i=1}^n t_i$. If the condition is hold, consider we do jobs at increasing order of b_i/t_i , then for every i , we have

$\frac{\sum_{j=1}^i b_j}{\sum_{j=1}^i t_j} \leq \frac{\sum_{j=1}^n b_j}{\sum_{j=1}^n t_j} \leq \frac{\sum_{j=i+1}^n b_j}{\sum_{j=i+1}^n t_j}$, so overflow will not occur at $\sum_{j=1}^i t_j$. During one job, obviously $rt - B$ is a linear function with slope $r - b_i/t_i$, so overflow will not occur at any time.

Prob 7 (P198 T19)

A group of network designers at the communications company CluNet find themselves facing the following problem. They have a connected graph $G = (V, E)$, in which the nodes represent sites that want to communicate. Each edge e is a communication link, with a given available bandwidth b_e .

For each pair of nodes $u, v \in V$, they want to select a single $u - v$ path P on which this pair will communicate. The bottleneck rate $b(P)$ of this path P is the minimum bandwidth of any edge it contains; that is, $b(P) = \min_{e \in P} b_e$. The best achievable bottleneck rate for the pair u, v in G is simply the maximum, over all $u - v$ paths P in G , of the value $b(P)$.

It's getting to be very complicated to keep track of a path for each pair of nodes, and so one of the network designers makes a bold suggestion: Maybe one can find a spanning tree T of G so that for every pair of nodes u, v , the unique $u - v$ path in the tree actually attains the best achievable bottleneck rate for u, v in G . (In other words, even if you could choose any $u - v$ path in the whole graph, you couldn't do better than the $u - v$ path in T .)

This idea is roundly heckled in the offices of CluNet for a few days, and there's a natural reason for the skepticism: each pair of nodes might want a very different-looking path to maximize its bottleneck rate; why should there be a single tree that simultaneously makes everybody happy? But after some failed attempts to rule out the idea, people begin to suspect it could be possible.

Show that such a tree exists, and give an efficient algorithm to find one. That is, give an algorithm constructing a spanning tree T in which, for each $u, v \in V$, the bottleneck rate of the $u - v$ path in T is equal to the best achievable bottleneck rate for the pair u, v in G .

We claim that the maximum spanning tree of G is actually the wanted tree. Thus we can use K algorithm to solve it in $O(n\alpha(n) + m \log m)$.

To proof the correctness, consider the maximum spanning tree is T . For u and v , the optimal path is $P' = u \leftrightarrow v$ and distinct to $P = u - v$ on T . Then we have $\min_{e \in P'} b_e > \min_{e \in P} b_e$. Let $e_0 = \arg \min_{e \in P} b_e$, and consider $T \setminus \{e_0\}$. Obviously T will split into 2 components called U and V , and suppose $u \in U, v \in V$ (If $u, v \in U$, then $U \cup \{e_0\}$ will form a ring). Since P' connects u and v , then there must exists a edge $(x, y) = e_1 \in P'$ that $x \in U$ and $y \in V$ (otherwise $P' \subseteq U$ or $P' \subseteq V$). We found that $T' = (T \setminus \{e_0\}) \cup \{e_1\}$ is a spanning tree that greater than T since $b(e_1) > b(e_0)$, contradiction.